

AixKer: High-Performance Networking Stack via eBPF & XDP

Technical Architecture & Design Principles

Author: Ali Heidari | **Project:** <https://aixker.com>

1. Executive Summary

AixKer is a next-generation networking stack built with **Rust**, leveraging **eBPF** and **Reinforcement Learning (RL)** to optimize packet processing. By bypassing the standard Linux kernel networking overhead and introducing intelligent, autonomous agents at the kernel level, AixKer achieves predictable low latency and dynamic load balancing for high-throughput infrastructure.

2. The Problem: Kernel Overhead & Static Routing

Modern cloud-native applications often face bottlenecks in the Linux kernel's networking path:

- **Context Switching & Copying:** Frequent switching between user/kernel space and memory copying consumes excessive CPU.
- **Dumb Load Balancing:** Traditional load balancers often use static algorithms (like Round Robin) that are unaware of the actual real-time stress on individual nodes, leading to uneven load distribution and latency spikes.

3. The AixKer Solution

AixKer addresses these inefficiencies through a hybrid architecture combining low-level optimization with AI-driven decision making:

3.1. XDP (eXpress Data Path) Integration

AixKer attaches an eBPF program directly to the network interface card (NIC) driver. This allows the system to make routing or dropping decisions **before** the OS kernel allocates memory for an `sk_buff`, significantly reducing overhead.

3.2. Zero-Copy Architecture via AF_XDP

The protocol utilizes `AF_XDP` sockets to establish a shared memory region (UMEM) between the kernel and the userspace application. This allows the Rust application to process data without costly memory copies.

3.3. AI-Driven Adaptive Load Balancing (The Core Innovation)

Unlike standard stacks, every AixKer node is equipped with an embedded **Intelligence Agent**:

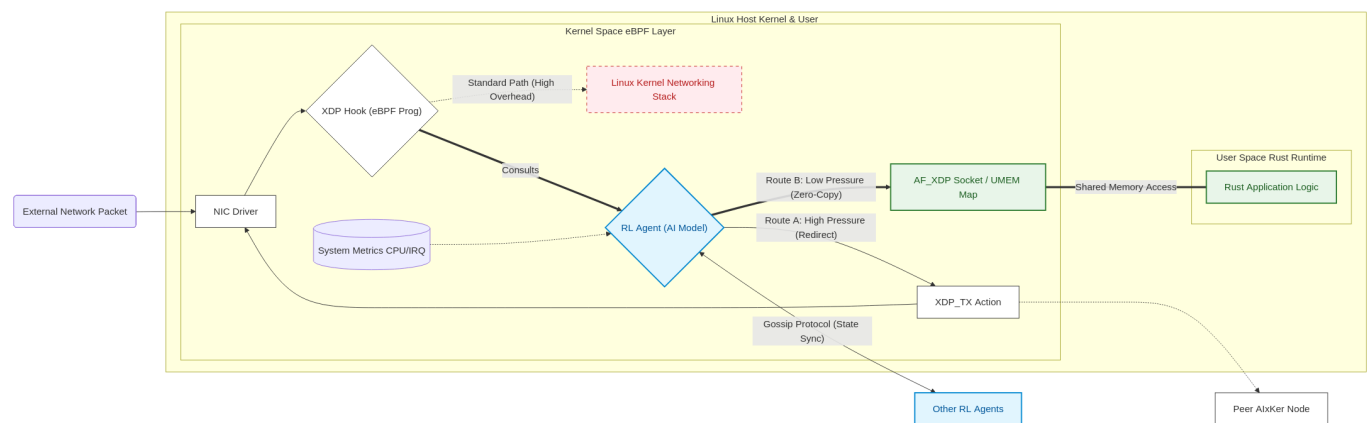
- **RL-Based Decision Engine:** Each agent utilizes a model trained via **Reinforcement Learning** to continuously evaluate the machine's state based on real-time metrics (CPU load, IRQ pressure, Memory usage).

- **Lightweight Gossip Protocol:** Agents discover and communicate with peer agents using a custom, high-speed/low-overhead network protocol to share state information.
- **Smart Redirection:** When a packet arrives at the kernel (XDP hook), the Agent evaluates the local node's pressure. If the node is stressed, the packet is instantly **redirected** to a peer agent identified as having lower load. This ensures optimal cluster utilization and prevents node saturation.

3.4. Rust Safety & Performance

The control plane and userspace logic are written in **Rust**, ensuring memory safety and concurrency without the latency spikes associated with Garbage Collectors.

4. Diagram



5. Key Performance Metrics (Benchmarks)

Internal benchmarking against the standard Linux network stack demonstrates significant improvements:

- **Latency:** Consistently achieving **sub-200µs** round-trip times (RTT).
- **Throughput:** Capable of handling millions of packets per second (PPS) on commodity hardware.
- **Stability:** The RL agent prevents tail latency spikes by shedding load *before* a node becomes unresponsive.

6. Use Cases

AlxKer is optimized for scenarios requiring extreme efficiency:

- **Real-time Streaming Platforms** (Intelligent alternatives to Kafka/Fluvio).
- **High-Frequency Trading (HFT)** infrastructure.
- **Edge Computing** clusters where nodes have varying capacities.

7. Future Roadmap

- Expanding the RL model to predict traffic bursts before they happen.
- Implementation of full userspace TCP stack logic.
- Hardware offloading support for SmartNICs.

For further technical details or a live demonstration of the Agent's redirection logic, please contact me directly.